Fieldbook — Product & Technical Overview

Oct 18, 2026 · @Pranav

Fieldbook is our system of record for teleoperation data-collection tasks: designers create and version tasks, mark them Ready, and our existing operator tool pulls them through a small API and reports episodes back. Phase 1 is built and tested locally; going live needs only cloud accounts and a domain.

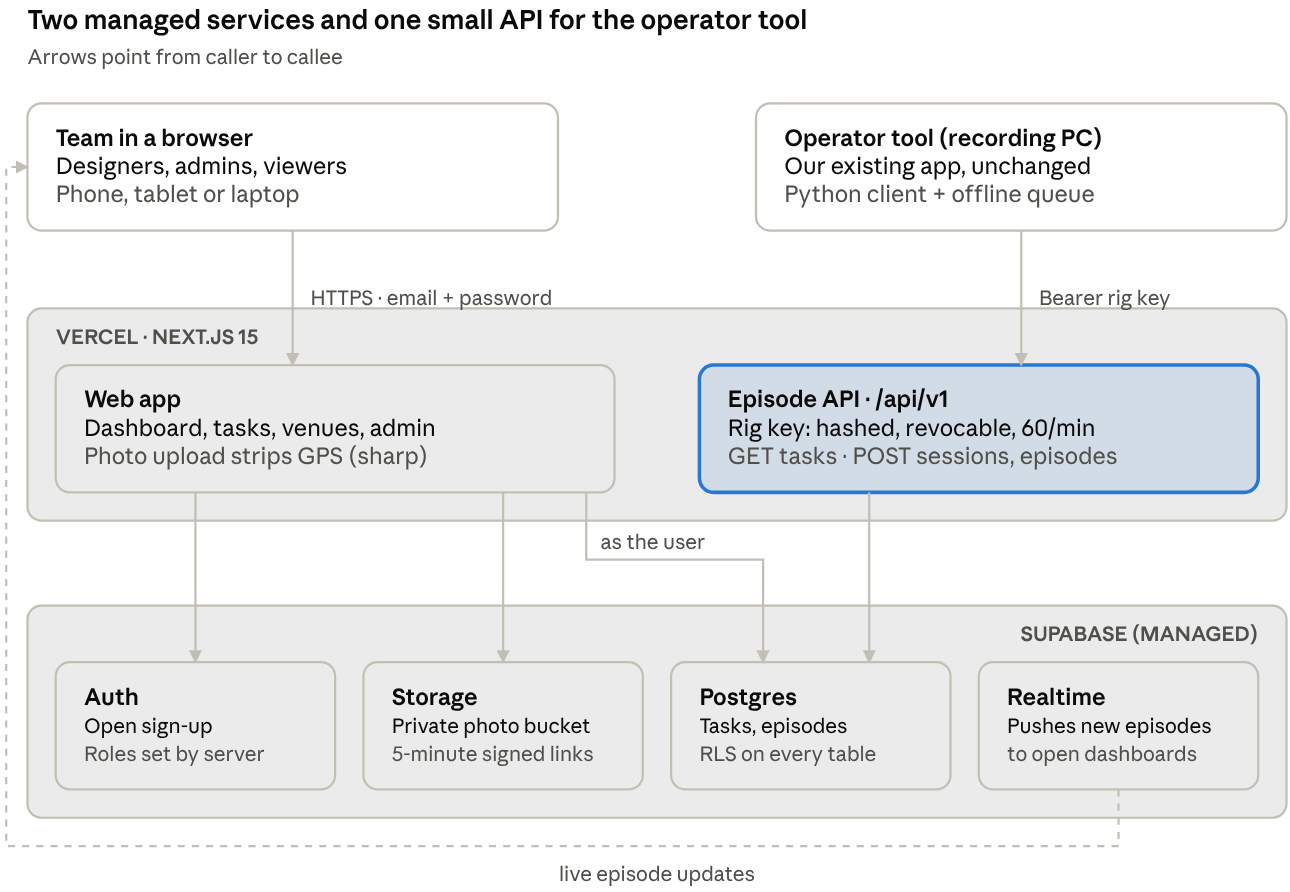
Problem and goals

The task list lived in one Google Sheet (~290 tasks across 2 venues) that couldn't tell us what to record next or how much data we had. Each problem maps to one fix:

Spreadsheet problem	Fieldbook fix
Scenes were implied by row position	Scenes are records with photos, clearance notes and reset steps
Edits overwrote history ("NEW ITERATION" typed into notes)	Every instruction change is a new version with a reason; episodes record their version
Feasibility was free text	A status workflow: Proposed → Feasibility test → Ready → Collecting → Done
Props mixed "already there" and "must buy"	Props are tagged at-scene vs. to-buy, rolled into a shopping list
"Done" was a checkbox	Episode counts per task against a target; Done is set automatically
~55 photos pasted in, tied to nothing	Photos mapped to scenes on import; the rest wait in an Unsorted inbox
Anyone with the link saw everything	Accounts, roles and per-venue access enforced in the database

Goals: one source of truth for tasks, tasks flow to operators without copy-paste, live progress toward data targets, and stealth-grade security from day one.

Architecture



Fieldbook architecture · who calls whom

Browser requests run as the signed-in user, so Postgres row-level security decides what each person sees. The operator tool never touches the database directly: only the Episode API, with a revocable key per recording PC. Stack: Next.js 15 on Vercel; Supabase for Postgres, auth, private photo storage and realtime updates.

Requirements

All Phase 1 requirements are built and covered by 66 automated tests; the remaining items are deployment steps, not features.

Functional

Requirement	Status
Sign in with any email + password; open sign-up	Built

Requirement	Status
New accounts view-only until an admin grants a role and venues	Built
Venues and scenes with photo galleries; phone-camera upload	Built
Tasks: create, edit, search, filter, bulk status change, CSV export	Built
Version history with change reasons and word-level diffs	Built
Duplicate-task warning while typing	Built
Props per task (at scene vs. to buy) and a printable shopping list	Built
Publish to operators: "Ready" switch on create, Move to Ready / back to Proposed	Built
Episode API for the operator tool + Python client with offline queue	Built
Dashboard: KPIs, 14-day chart, pipeline, venue progress, live activity	Built
Admin: users, roles, venue assignment, deactivation, rigs, API keys, audit log	Built
Spreadsheet import (both sheets merged and deduped, photos mapped)	Built

Non-functional

Requirement	How it is met
Security: per-venue access, no data leaks	Row-level security on every table (37 tests); private photos; GPS stripped
Stealth	Neutral app name, no search indexing, no third-party analytics or fonts

Requirement	How it is met
Devices	Responsive from 390 px phones to 1440 px projectors; light and dark mode
Scale	Sized for 15–50 users; managed Postgres scales well past that
Reliability	Idempotent episode API: client retries never create duplicates
Hosting	Vercel + Supabase, US region; daily backups (Supabase Pro)

Roles and access

Four roles, enforced inside Postgres: hiding a button is never the only protection.

Role	Sees	Can do
Admin	Everything	Manage users, roles, venues, rigs, API keys; read the audit log; everything a designer can
Designer	All venues	Create and edit tasks, scenes, props; change status; export CSV
Operator	Assigned venues only	Upload scene photos; record feasibility results; log sessions and episodes
Viewer (every new sign-up)	Assigned venues only (none at first)	Read only

Roles are set only by the server, so a sign-up can never make itself admin. Deactivating someone bans the account and revokes every session immediately; their history stays attributed. Every write is recorded in an admin-only audit log.

Operator tool integration

Our existing operator tool stays as it is; it needs two calls to fetch Ready tasks and report episodes. Each recording PC gets its own API key (`rig_<prefix>_<secret>`), stored hashed, shown once, revocable, limited to 60 requests a minute.

Endpoint	Purpose
`GET /api/v1/tasks`	Ready and collecting tasks: code, instruction, version, scene, photo link (5 min), props, episodes remaining
`POST /api/v1/sessions`	Start a session for an operator at a venue
`POST /api/v1/episodes`	Log one episode (success / failure + reason, duration, recording location). Safe to retry: same ID never duplicates
`PATCH /api/v1/sessions/{id}`	End a session
`GET /api/v1/health`	Check the key works

A ready-made Python client (only depends on `requests`) wraps these calls and queues episodes on disk when venue Wi-Fi drops. Recordings themselves stay wherever they live today; Fieldbook stores only a pointer to each file.

Data model

Everything hangs off venues: a venue has scenes, a scene has photos and tasks, and episodes link a task version to the session that recorded it.

Record	What it holds
Venue	Name, address, photo-folder link; which people are assigned to it
Scene	Physical setup at a venue: description, surface height, clearance notes, reset steps
Scene photo	Reference / layout / reset photos; private, GPS stripped, with thumbnail

Record	What it holds
Task	Permanent code (T-0042), instruction, status, arms required, difficulty, weight limit, target episodes, success count, tags
Task version	Every instruction ever used for a task, who changed it and why
Prop	Shared catalogue; per task: quantity, at scene or to buy, bought yet
Rig + API key	A recording station (arm, headset, arm count) and its revocable keys
Session	One operator at one venue on one rig, start to end
Episode	Outcome, failure reason, duration, recording location, task version
Audit log	Who changed what and when, for every write

Deployment and running cost

Going live takes about a day of setup and roughly $45 a month; there are no servers to run.

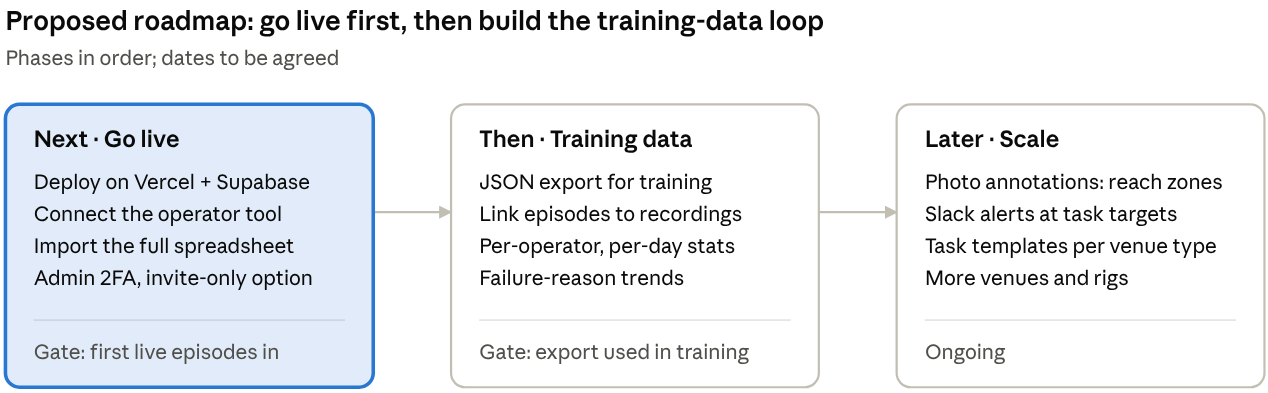
1. Buy a neutral domain.
2. Create a Supabase Pro project (US region) and push the database schema.
3. Turn on sign-up, password rules and leaked-password protection; optionally connect an email provider for email confirmation.
4. Deploy to Vercel Pro and add the environment keys; enable preview protection and a rate limit on the API.
5. Sign up, promote the first admin, create rigs and one API key per recording PC.
6. Import the full spreadsheet.

Item	Monthly cost (approximate)
Supabase Pro (database, auth, storage, backups)	~$25
Vercel Pro (one seat)	~$20
Domain	~$1

Item	Monthly cost (approximate)
Email provider (only if email confirmation is on)	$0–$20

Prices are list prices from memory, not checked against current pricing; confirm before budgeting.

Future scope



Proposed roadmap · 3 phases

The next phase is about going live and connecting the operator tool; after that, the value comes from turning episodes into a training-ready dataset. Order and scope are a proposal for the founders to set.

Risks and open questions

Risk	Mitigation
Open sign-up attracts bots or curious outsiders	New accounts see nothing; sign-up rate limits; leaked-password check; can switch to invite-only by one setting
Passwords only (no 2FA)	Strong-password rules now; add 2FA for admins later (Supabase supports it)
Operator tool integration effort unknown	API is two calls; Python client ready; integration is the first next step

Risk	Mitigation
Source spreadsheet quality (e.g. a misplaced photo on one row)	Import is re-runnable; Unsorted inbox and photo moving fix mistakes in the UI
Vendor lock-in	Standard Postgres; data exports to CSV; schema lives in our repo

Open questions:

- Which recording stack does the operator tool use (LeRobot, ROS 2, custom)? It decides the client adapter.
- Where should recordings live (local, S3, GCS)? Fieldbook only stores the pointer.
- Is 50 successful episodes the right default target per task?
- Should sign-up stay open, or move to invite-only before outside contractors join?